Mini-Project

Milan Lagae

University Name: Vrije Universiteit Brussel

Faculty: Sciences and Bioengineering Sciences

Course: GPU Computing

March 29, 2026

Contents

1. Intro

The first section involves a direct comparison between addition and multiplication operations to establish a baseline, as specified in the assignment descriptions.

Following the comparison in the first part, the kernels are modified to execute on multiple elements simultaneously in two different one-to-many mapping's schemes [1].

For the third section, the compute intensity of the benchmark are parameterized by using a loop count factor. Increasing the compute intensity should surface the limitation of the memory & compute of the GPU.

The final section will compare the trade-offs between local vs global memory on the performance of the execution, as indicated in the schema [2] in the assignment.

2. Structure

A `src` folder is included in the zip file. This folder contains a `project-desktop` folder, the kernel files and the CMake project file. The `src` folder contains all the `c++` files responsible for orchestrating the benchmark execution. The following list of kernels are included: `partOne`, `partTwo`, `partThree`, `partThree-2` and `partFour`.

3. Methodology

A total of **20** runs were performed for each parameter combination. The first 5 runs were discarded to allow the system to stabilize. This is in accordance with the recommendations in [3]. For each measured value, if applicable, a CoV range chart is produced and will be referenced. These can be found in Section 9. The acceptable range of the CoV is taken from [4].

4. Part 1: Addition vs Multiplication

This section evaluates the performance differential between addition and multiplication operations. Furthermore, it quantifies the overall GPU acceleration relative to a CPU baseline to determine the magnitude of the observed speedup

4.1. Setup

The code for this part can be found in the file: `part-1.cpp` and the kernel file in `partOne.cl`. Included in the `partOne.cl` kernel file are two separate kernels, respectively: `mul_continuous` and `add_continuous`.

4.2. GPU Analysis

The evaluation begins with an analysis of GPU performance, utilizing execution time as the primary metric. Figure 1 illustrates the relationship between execution time and array size, with distinct colors utilized to differentiate between the addition and multiplication operations.

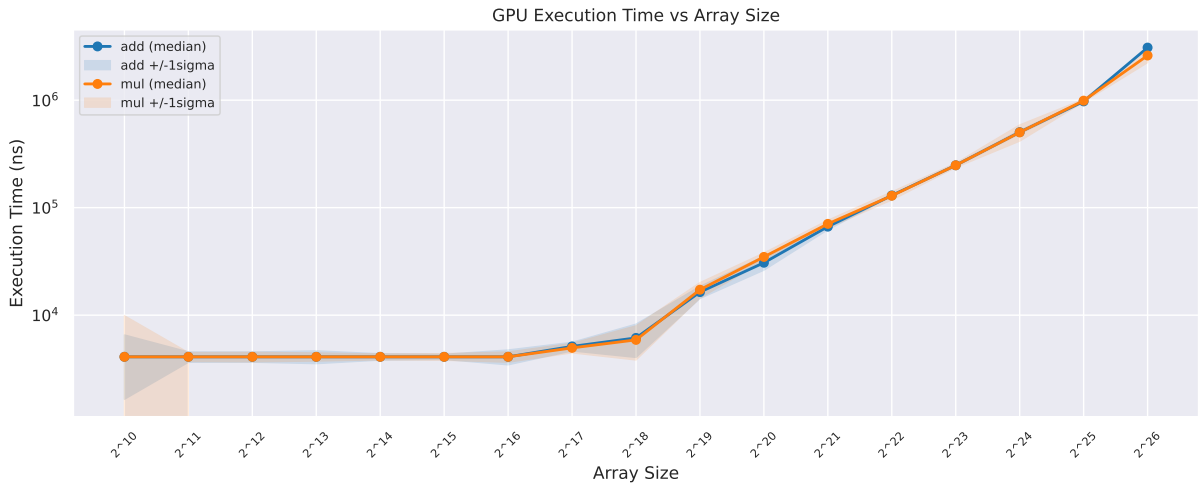

Figure 1: GPU Execution time vs Array Size

A notable observation from the data is that the computational workload only becomes significant enough to impact execution time at an array size of 2^{18}. Beyond this threshold, the execution time scales linearly with the array size, as evidenced by the constant slope on the logarithmic plot.

To further evaluate performance, the bandwidth utilization is plotted against array size in Figure 2.

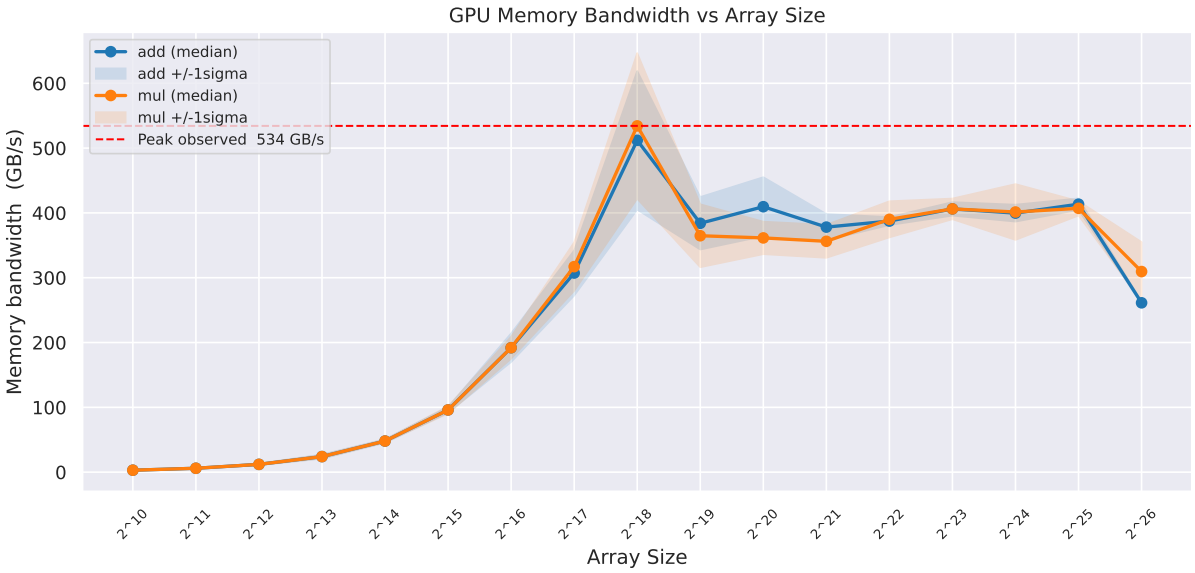

Figure 2: GPU Memory Bandwidth vs Array Size

The effective memory bandwidth scales proportionally with array size until reaching a peak at 2^{18}. Beyond this saturation point, throughput begins to decline as the system encounters overheads associated with kernel invocation, kernel inefficiencies and hardware-level memory constraints. At scales of 2^{26} and higher, the results suggest that the GPU enters a memory-bound regime, where performance is strictly limited by the available bus width.

To provide a more robust basis for these conclusions, the arithmetic throughput is examined in Figure 3. This metric allows for a precise evaluation of the computational intensity across varying array sizes.

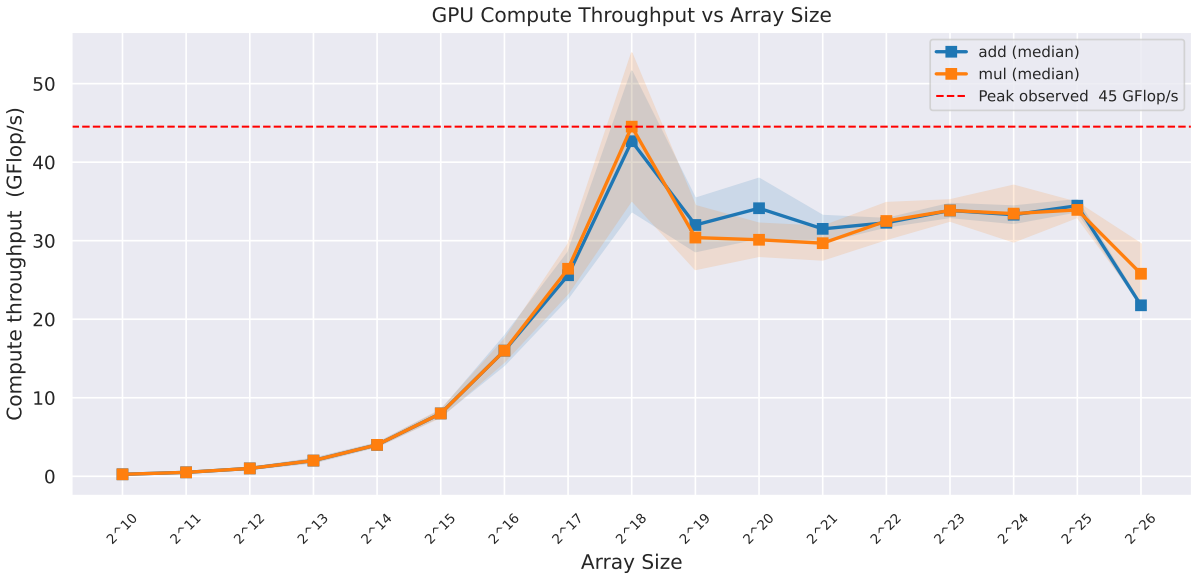

Figure 3: Compute Throughput vs Array Size

The trends observed in Figure 3 mirror those in Figure 2. A primary conclusion derived from these figures is that the current kernel implementation is memory-bound, as performance scales in direct correlation with memory bandwidth utilization, rather than raw computational capacity.

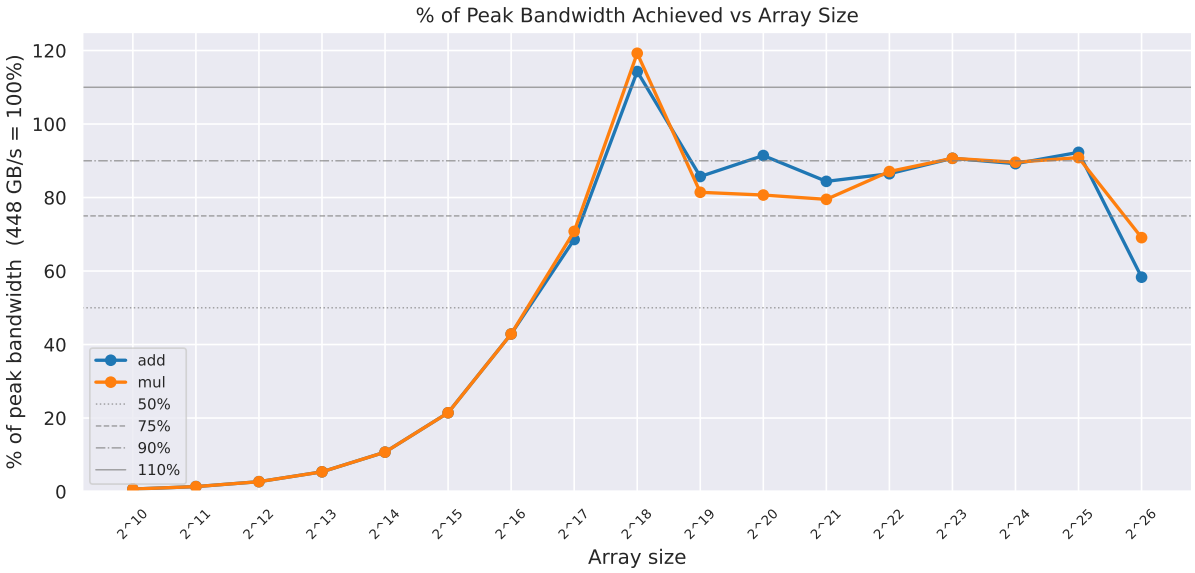

Figure 4: Bandwidth Percentage Utilization vs Array Size

Normalizing the observed bandwidth against the GPU's theoretical peak confirms that the implementation is bandwidth-limited as illustrated in Figure 4. The throughput maximum at 2^{18} **may** suggest optimal cache utilization. For the RTX 3070 architecture, the datat size aligns with the 4MB L2 cache limit[1]. Beyond this threshold, the working set exceeds cache capacity, forcing the system to rely on global memory bandwidth.

4.3. CPU vs GPU

A comparative analysis of GPU versus CPU performance is presented in Figure 5. This comparison quantifies the GPU speedup by contrasting the high-throughput parallel architecture of the GPU against the sequential execution model typical of a CPU.

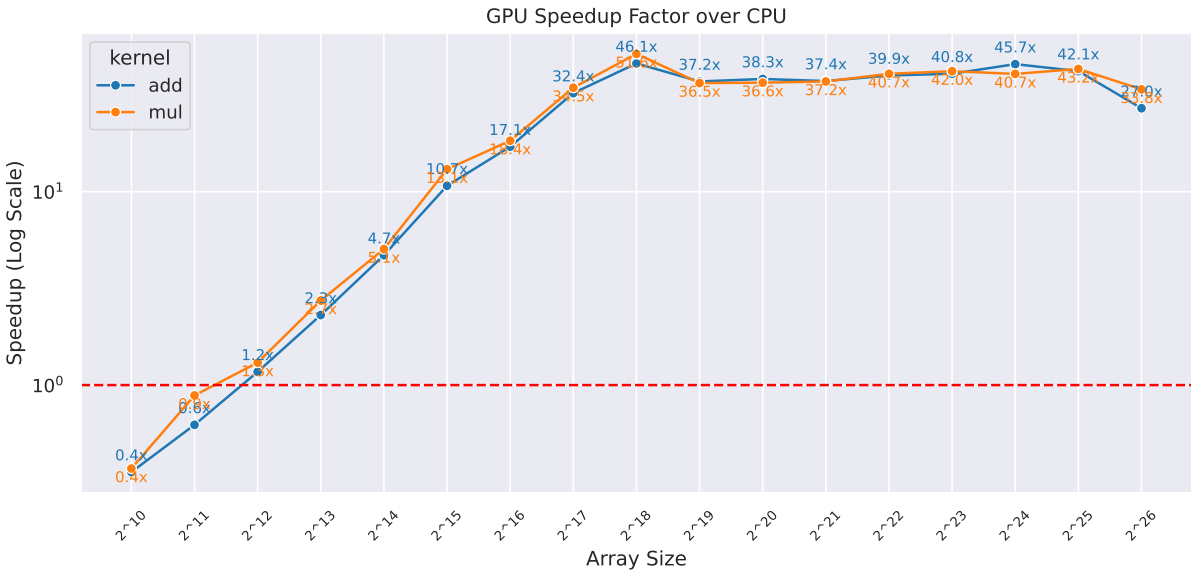

Figure 5:

[1]https://www.techpowerup.com/gpu-specs/geforce-rtx-3070.c3674°

The CPU performance is benchmarked using a baseline sequential implementation consisting of a standard iterative loop. Further optimization; such as multithreading or SIMD, could enhance CPU throughput. For this part, the CPU will keep using a naive algorithm.

4.4. Conclusion

The benchmarks reveal no significant performance difference between addition and multiplication operations. This symmetry in execution time suggests that both operations are equally optimized within the GPU's functional units. As illustrated in Figure 5, the parallel architecture of the GPU provides a substantial performance advantage over the CPU baseline.

5. Part 2: Elements Per Thread

This section analyzes the result of increasing the number of Elements Per Thread (EPT); a single thread (work-item) is responsible for on the performance of the execution. These different access patterns are described as `contiguous` and `strided`. They are based on the pdf [1] in the assignment description.

5.1. Setup

The array size is standardized at 2^{22} elements, a value derived from the benchmark results observed in Section 4. Furthermore, the workgroup size is fixed at 64 to maintain consistent results.

The benchmark file is called: `part-2.cpp`, and kernel file `partTwo.cl`. Included in the kernel file are the following kernels: `mul_contiguous`, `add_continguous`, `mul_strided` and `add_strided`.

The `contiguous` pattern, corresponds to the example code shown in Listing 1.

```
1 // host code
2 unsigned int data_index[W];
3 cl::NDRange global(W/N);
4 // device code
5 for (int i = 0; i < N; ++i)
6     data_index[N * get_global_id(0) + i] = get_global_id(0);
```

Listing 1: Example: Contiguous Pattern Code

The `strided` access pattern, corresponds to the example code shown in Listing 2.

```
1 // host code
2 unsigned int data_index[W];
3 cl::NDRange global(W/N);
4
5 // device code
6 for (int i = 0; i < N; ++i)
7    data_index[get_global_id(0) + i*get_global_size(0)] =
  get_global_id(0);
```

Listing 2: Example: Strided Pattern Code

5.2. Visualization

The impact of the EPT variable on the workload distribution across individual work-items is illustrated in Figure 6.

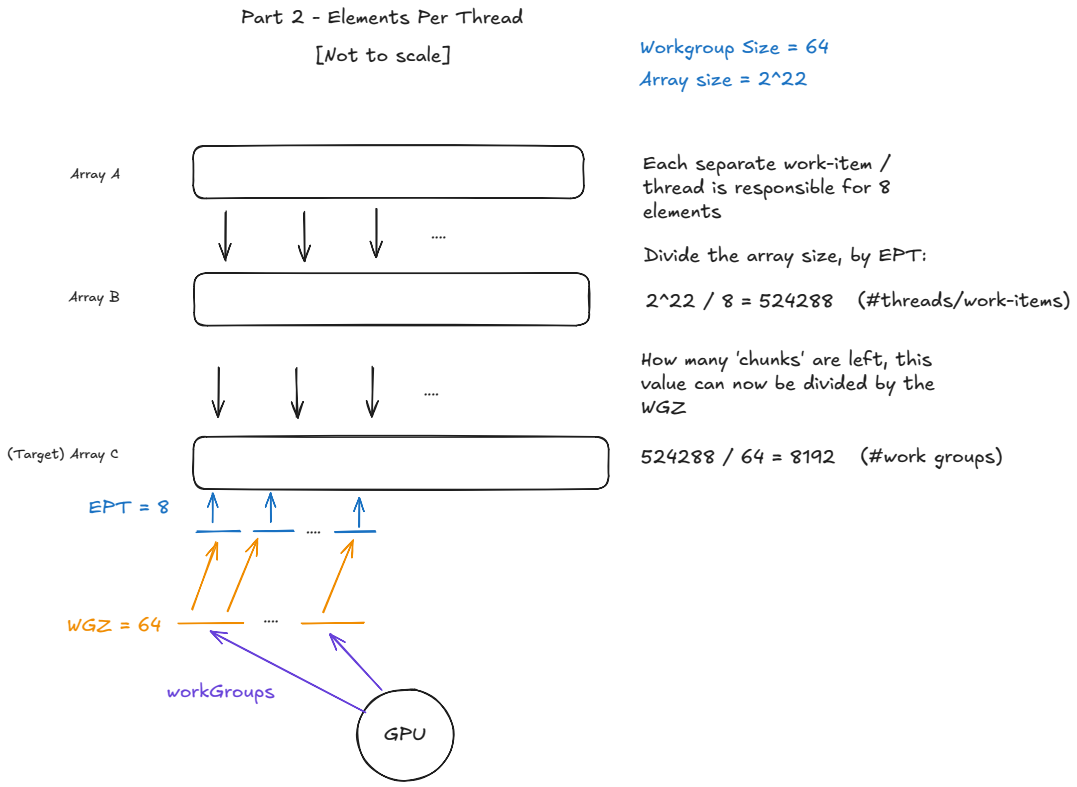

Figure 6: Elements Per Thread (8) - Work Item Orchestration

In this specific scenario, the EPT is set to 8, indicating that each individual work-item is responsible for processing eight elements from the source arrays. Consequently, the total number of work-items launched to cover the 2^{22} array size is reduced by a factor of eight, resulting in a global work size of 524, 288. Calculating the number of work groups can than be done, by dividing this number by the workgroup-size (64), which results in 8192 work groups.

5.3. Contiguous vs Strided

The initial analysis focuses on the execution time across both arithmetic operations, as well as the impact of the different access patterns. These relationships are illustrated in Figure 7, which plots execution time as a function of the EPT.

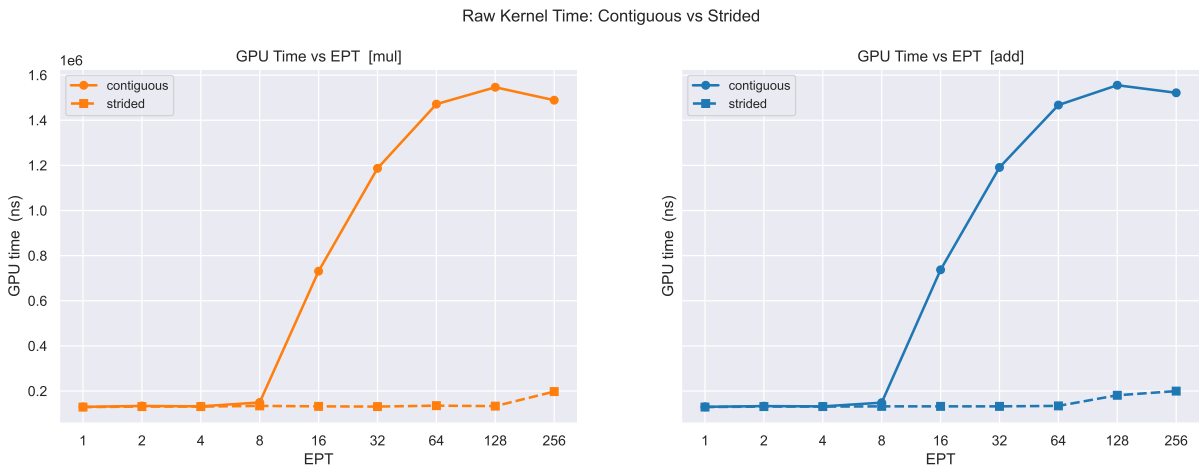

Figure 7: GPU Execution Time vs EPT - multiplication & addition operations - contiguous vs strided pattern

Initial observations from the chart suggest that the strided access pattern yields better performance compared to the contiguous pattern. This performance difference is further clarified in Figure 25, where a direct comparison of the two strategies reveals a significant speedup favoring the strided approach, as the EPT factor increases.

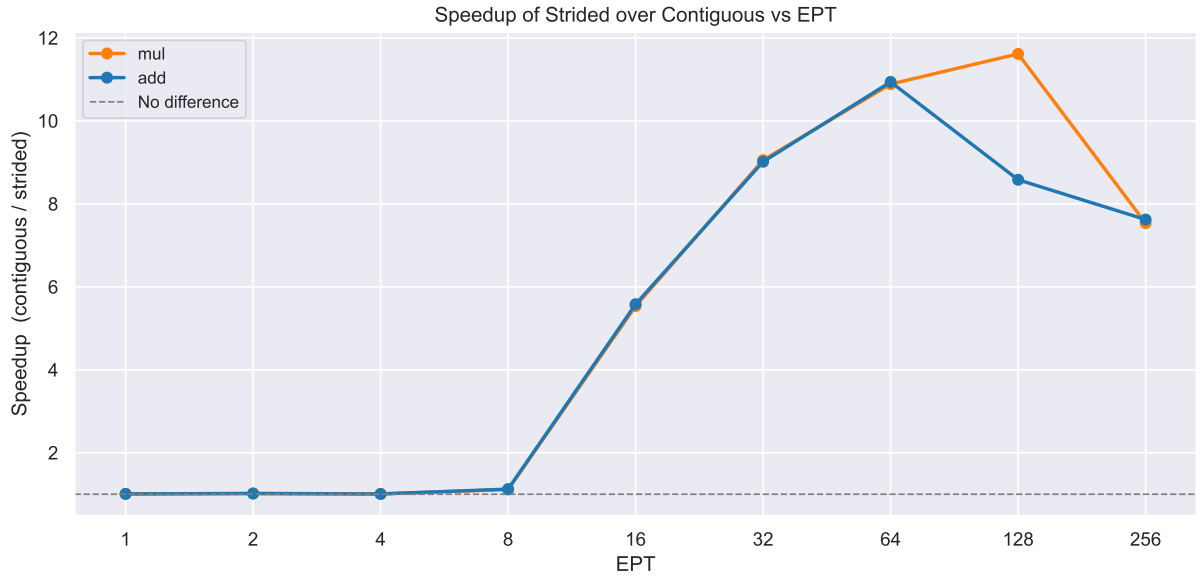

Figure 8: Strided Pattern Speedup vs Contiguous Pattern

The results demonstrate a performance inflection point for the strided access pattern once the EPT reaches a value of 8. The underlying mechanism for this acceleration can be assigned by the memory bandwidth analysis in Figure 9, which reveals a significant increase in data throughput compared to the contiguous implementation.

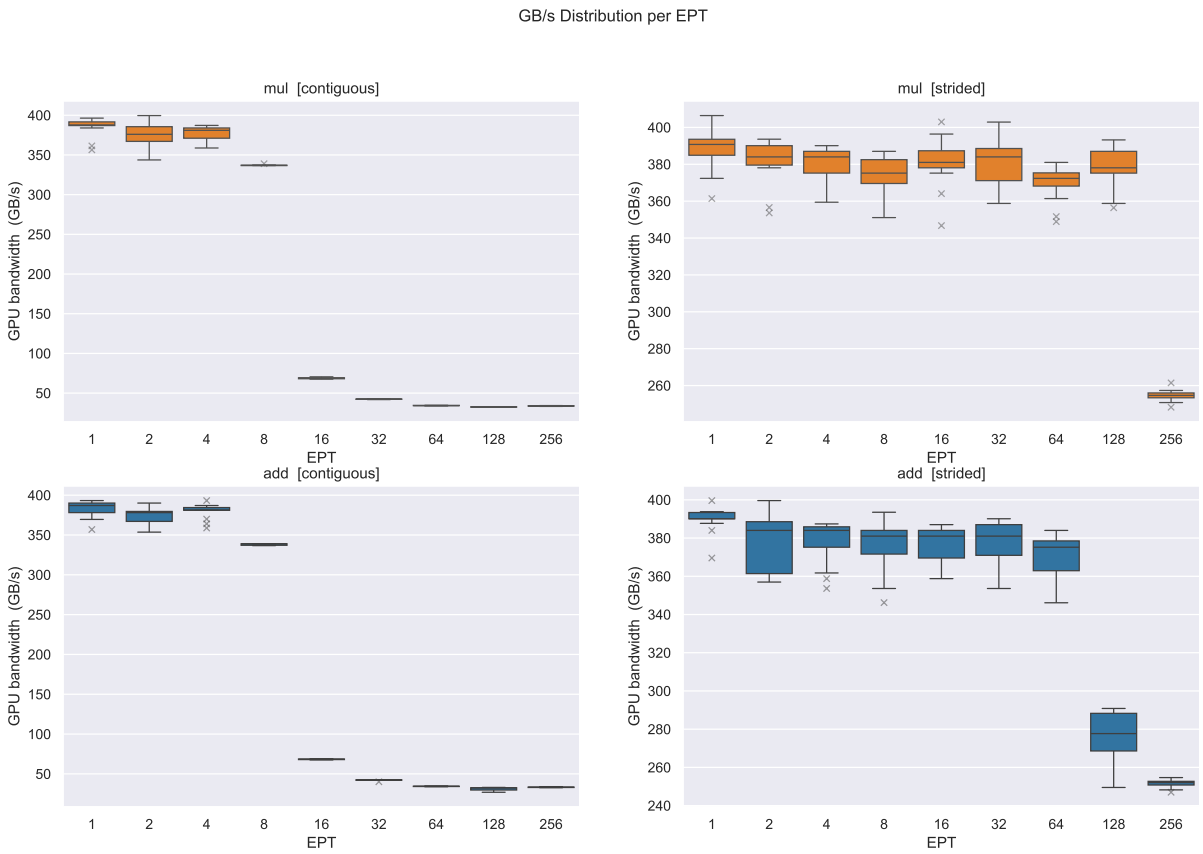

Figure 9: Memory Bandwidth CI Intervals

The contrast between the contiguous access pattern (left) and the strided pattern (right) is stark. A significant drop in memory bandwidth is observed for the contiguous model as the EPT increases from 8 to 16. Similar performance regressions occur at higher values, specifically when transitioning from 64 to 128 and 256, suggesting a potential bottleneck in the access pattern or GPU limitation.

The following analysis examines the computational intensity of both strategies, as illustrated in Figure 10, by comparing the arithmetic throughput across different EPT values.

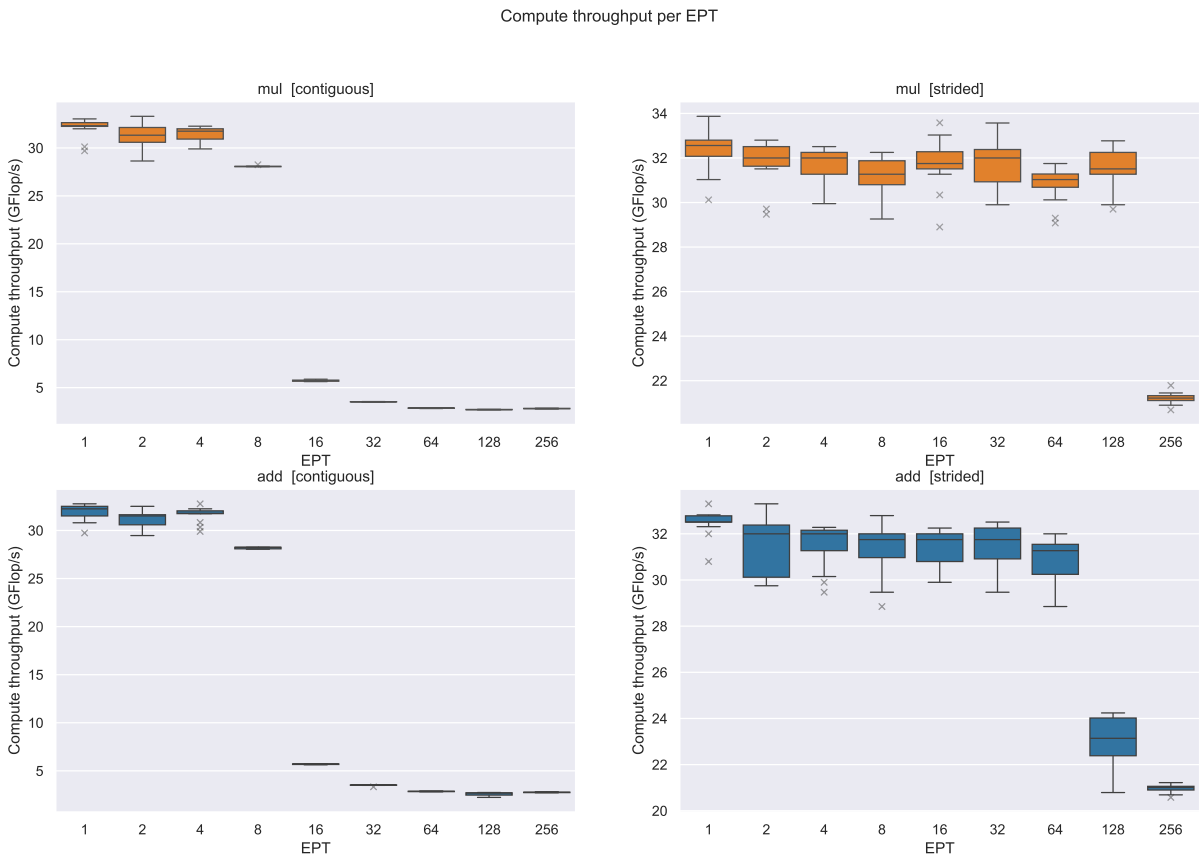

Figure 10: Computational Throughput CI Intervals

The data reveals a strong correlation between memory bandwidth and computational throughput. As the bandwidth utilization undergoes a sharp drop, the compute performance suffers an identical regression.

This synchronization confirms that the kernel is strictly memory-bound. Notably, while the contiguous and strided patterns exhibit these drops at similar thresholds, the multiplication operation displays a one-step phase shift in its performance decline compared to addition.

5.4. Conclusion

In summary, the data suggests that the strided access patterns provides the GPU with the data it needs to perform computations, up to the point where the number of EPT becomes too high. The GPU is unable to saturate the bandwidth and provide the needed data to the compute elements. At this point, the increased workload per thread likely exhausts local resources, preventing the GPU from effectively saturating the available bandwidth.

Furthermore, a notable disparity exists in the confidence intervals (CI); the contiguous pattern exhibits significantly lower variance than the strided approach. No immediate answer can be given for this difference in intervals between access patterns.

6. Part 3: Roofline Model

This section will discuss the experiment to create a roofline model as described in the assignment & discussed in the lectures. Before showcasing the roofline model, several other charts will be discussed first, showcasing the utility of the roofline model.

6.1. Setup

The array size is standardized at 2^{22} elements, a value derived from the benchmark results observed in Section 4. Furthermore, the workgroup size is fixed at 64 to maintain consistent results.

The benchmark file is called: `part-3.cpp`, and kernel file: `partThree.cl`. Included in the kernel file is a single kernel called: `float_sum_increasing_ci`. It combines the elements per thread loop with the kernel `intSumIncreasingCI`, present in the list of GPU exercises (given at the start of the semester). The benchmark file also updates the formulas for calculating the bandwidth (throughput), compute intensity (flops), and arithmetic intensity with formulas from the `sumIntsIncreasingCI.cpp` file.

6.2. Analysis

Figure 11 illustrates the relationship between the internal kernel loop count and three key indicators: compute intensity, memory bandwidth, and total execution time.

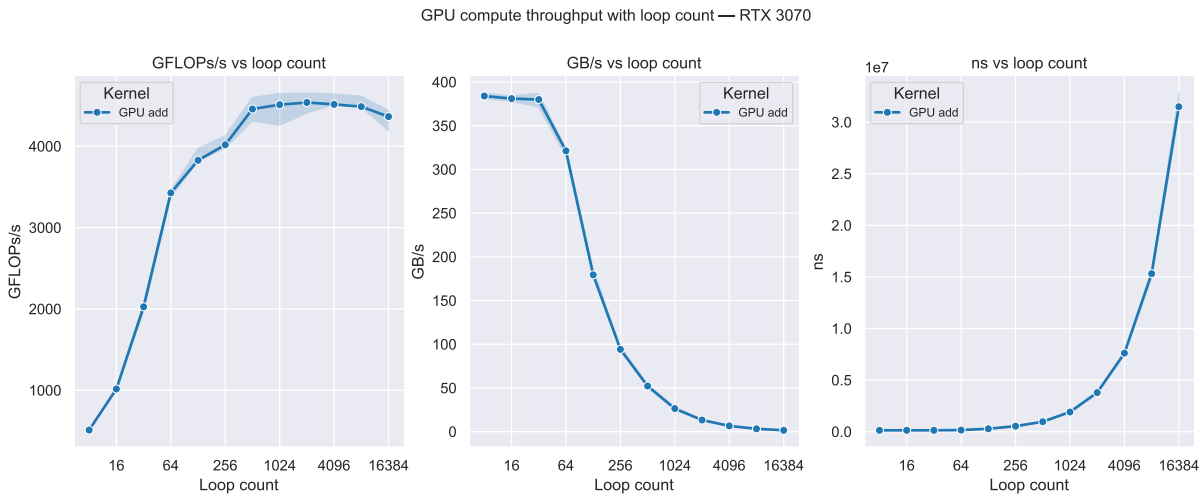

Figure 11: Loop Count vs Compute; Bandwidth & Execution Time

Execution runtime increases proportionally with the loop count. More significantly, the data reveals shift between compute intensity and memory bandwidth. This behavior is a key focus of the experiment, as it illustrates the GPU's shift from a memory-bound to a compute-bound regime.

Before analyzing the roofline model, Figure 12 illustrates how the computational intensity scales as the instruction density increases, providing a baseline for identifying the transition to a compute-bound state.

Figure 12: GPU Compute Throughput vs Loop Count

Increasing the EPT factor beyond a certain point (≥ 32) has a negative effect on the computational throughput.

A corresponding analysis for effective memory bandwidth is presented in Figure 13, where the throughput is parameterized by both the loop count and the EPT factor.

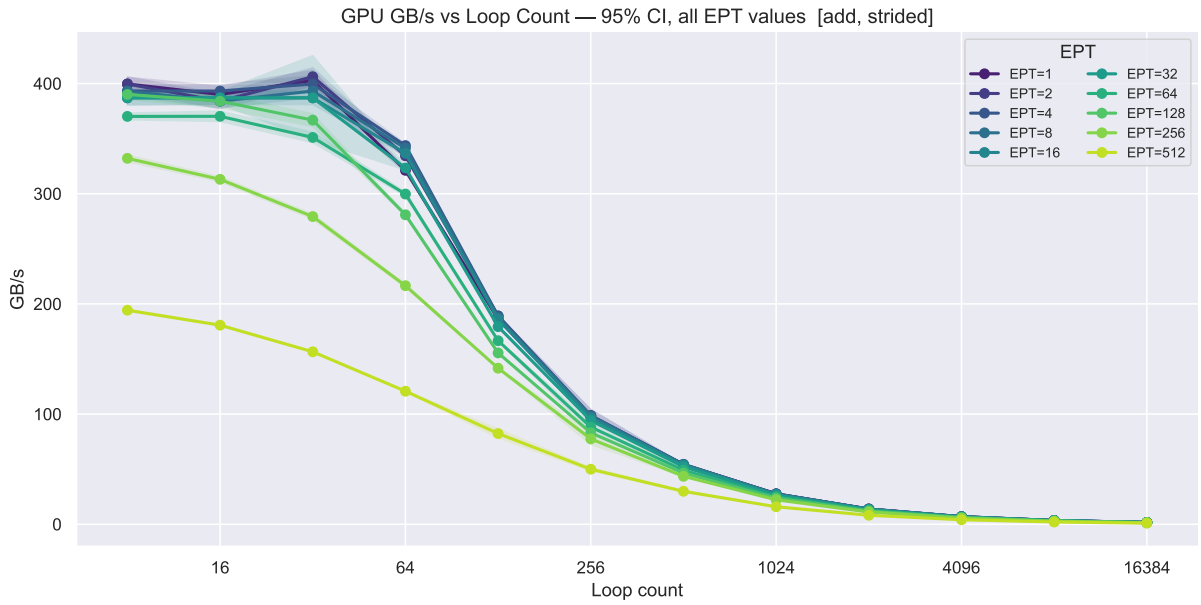

Figure 13: Memory Bandwidth Utilization vs Loop Count

The same behavior as seen in graphs in figure Figure 11, is again visible in the above two graphs, for different loop counts. The EPT value 'only' determines the ranges of the values, but does not change the chart behavior.

Increasing the EPT beyond (≥ 32) for any loop factor also showcases a decreasing bandwidth and computational throughput. Potentially indicating that due to the smaller number of threads / work-items launched, the GPU cannot fully utilize its compute and bandwidth capabilities.

Figure 14 presents the Roofline Analysis, which summarizes all the plots and behaviors described earlier.

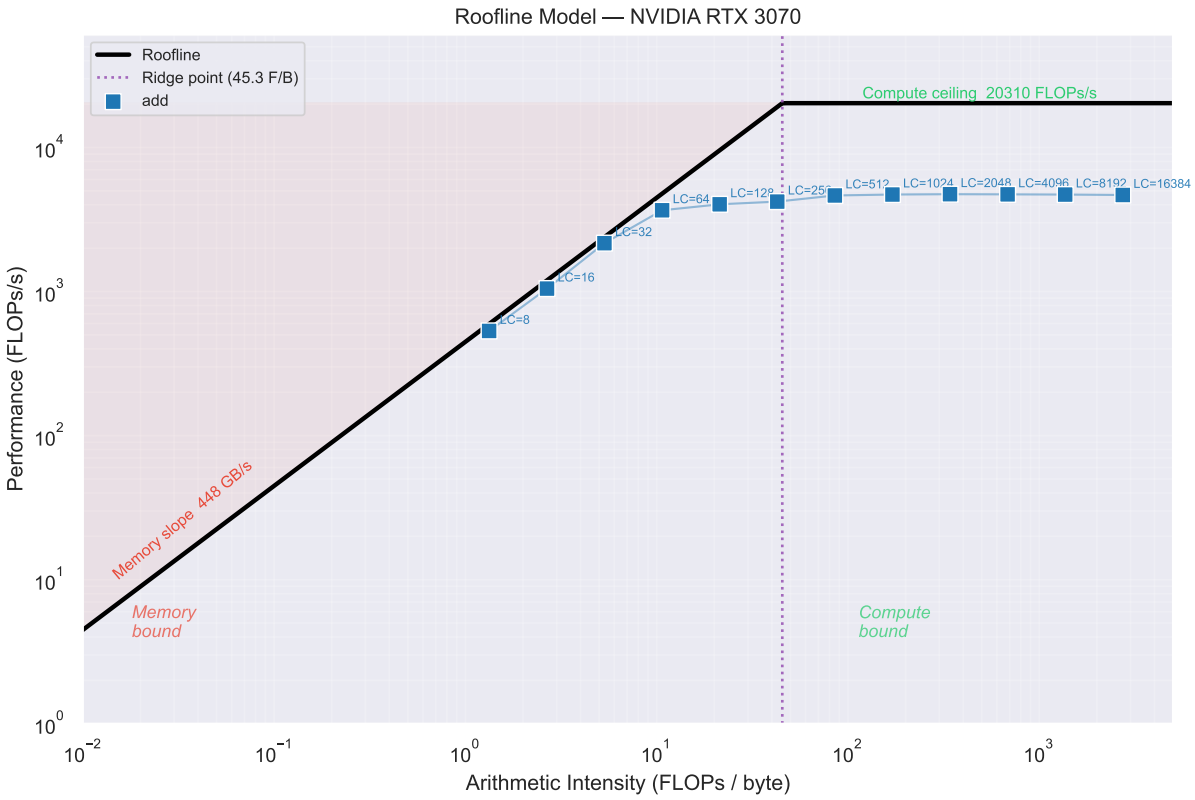

Figure 14: RTX3070 - Roofline Model

For the GPU in question, the memory bandwidth & peak computational throughput is indicated. The results follow the memory slope from the starting loop count (LC) 8 until 64, where it starts diverging. The 'ridge point', at which the actual data starts becoming 'memory bound', occurs much earlier than the theoretical value.

6.3. Conclusion

The experiment in question has clearly shown the appearance of the roofline model in the data. This part of the experiment can be considered a success, with a small remark made for the practical ridge point not matching the expected theoretical point.

7. Part 3.2: Workgroup Size

This section evaluates the sensitivity of the Compute Intensity (CI) kernel from part 3, to variations in workgroup size.

7.1. Setup

The array size is standardized at 2^{22} elements, a value derived from the benchmark results observed in section Section 4. The benchmark file is called: `part-3-2.cpp`, and kernel file: `partThree.cl`. The same kernel as in section Section 6 is reused.

7.2. Analysis

Figure 15 provides a multi-dimensional analysis of execution time, utilizing a ridge line[2] visualization to map performance distributions across a range of Loop Counts (LC) and Elements Per Thread (EPT) values.

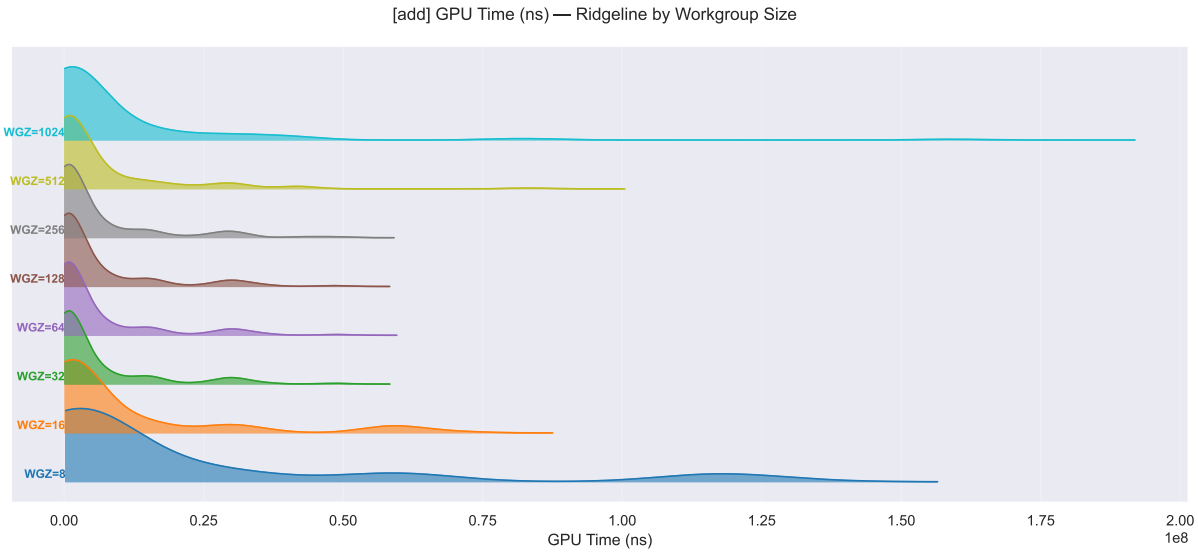

Figure 15: Ridgeline Visualization of the GPU time in function of workgroup-size

The ridge line analysis in Figure 15 indicates that the most optimal range of WGZ for the kernel in part 3 is between 32 — 256. This validates the choice to fix the WGZ to 64 across the benchmarks in part 1, 2, 3 & 4.

Completing this section with an additional roofline model chart as illustrated in Figure 16. The distribution of the charts for the different workgroup-size values is clearly visible.

[2] https://www.data-to-viz.com/graph/ridgeline.html

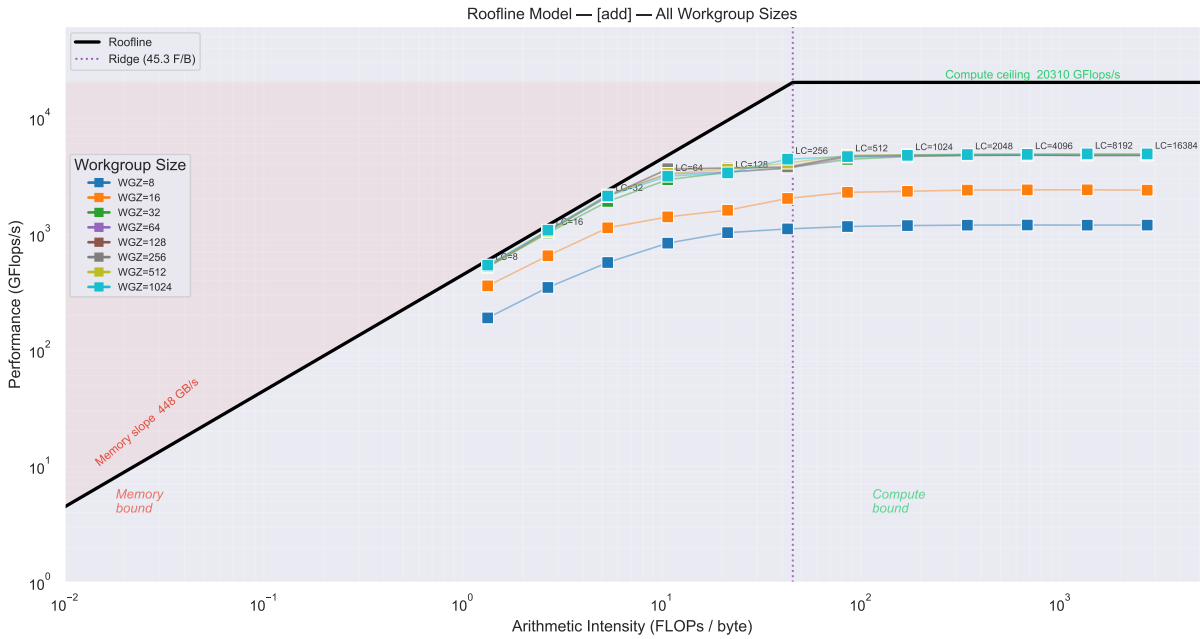

Figure 16: Roofline Model vs Workgroup Size

8. Part 4: Local vs Global

This section will discuss the performance difference between the use of local & global memory for a reduction kernel.

8.1. Setup

The array size is standardized at 2^{22} elements, a value derived from the benchmark results observed in Section 4. Furthermore, the workgroup size is fixed at 64 to maintain consistent results.

The benchmark file is called: `part-4.cpp`, and kernel file: `partFour.cl`. Included in the kernel file are two separate kernels: `add_reduction_global` and `add_reduction_local`. These kernels were taken from the exercise set. No modification were made to the kernels. The benchmarking file was modified to incorporate changes required for executing the benchmarks. Such modifications include; create src array based on array size, rewrite the src buffer on each GPU kernel run and other changes to fit into the benchmark harness.

8.2. Analysis

Figure 17 illustrates the difference between local & global memory pattern and three key indicators: compute intensity, memory bandwidth, and total execution time.

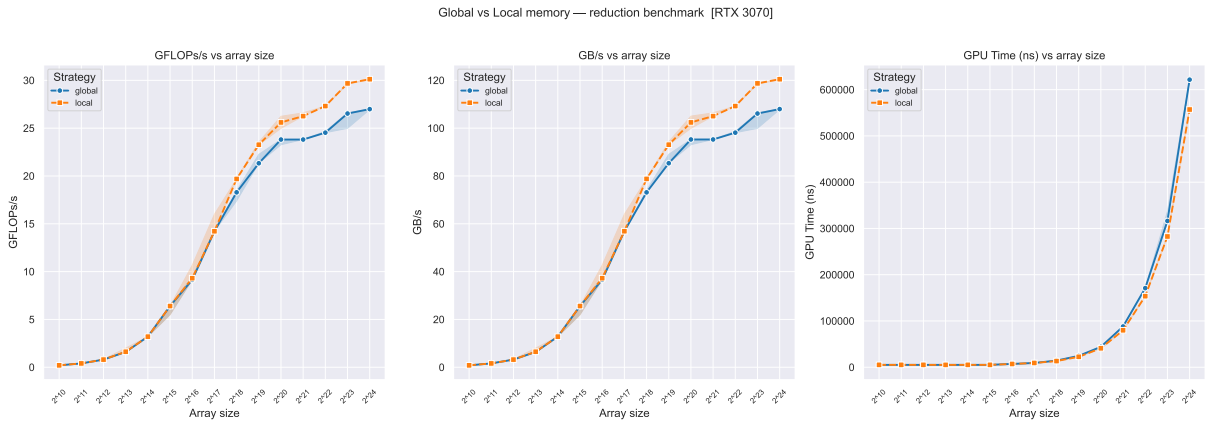

Figure 17: Global vs Local - GPU Time, Compute & Memory Bandwidth

Looking at the execution time on the right chart, it can be noticed that the execution time for the local memory pattern is slightly slower compared to the global pattern on larger sizes. The patterns start diverging with small amounts from array size $\geq 2^{20}$.

This reduction in execution time is reflected when looking at the memory bandwidth (middle) and compute throughput (left) charts. In both cases, the compute throughput and memory bandwidth are higher on larger array sizes and start widening from array size 2^{20}.

It should be noted that the behavior of the bandwidth & compute metric between both memory strategies follows a similar trend on larger array sizes. The global strategy does not reach the same peaks as the local strategy.

Plotting the execution time of both patterns against each other as illustrated in Figure 18.

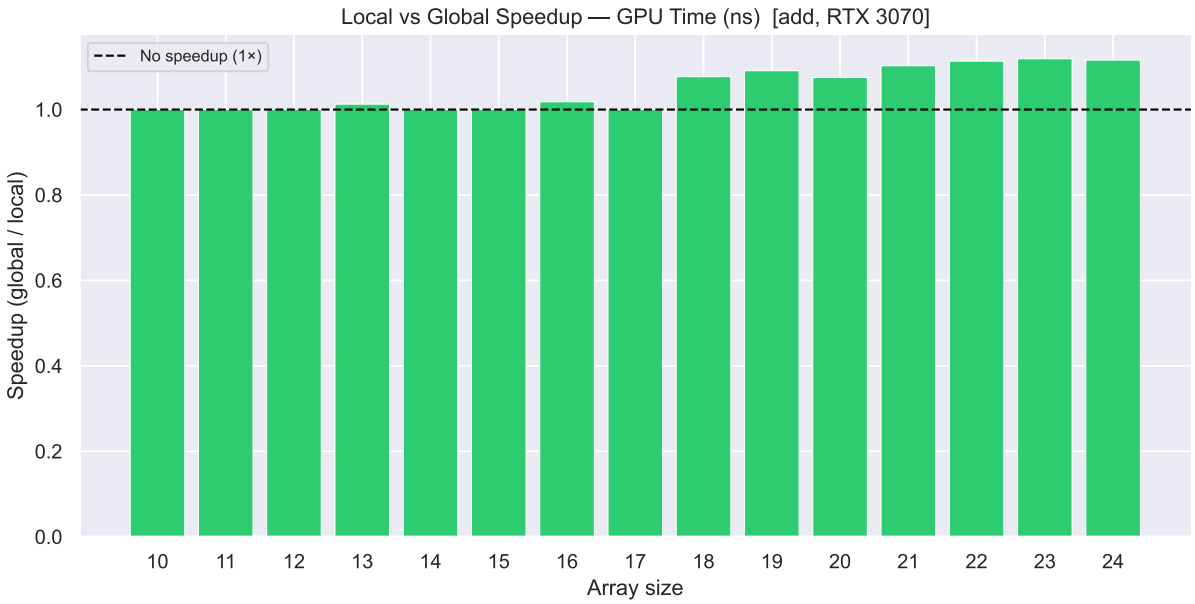

Figure 18: Local vs Global Speedup

The speedup alluded to earlier, is clearly visible starting from the array size 2^{18} and propagates until the end of the benchmarked array sizes. The peak speedup is measured to be around ~ 1.2 at an array size of 2^{23}.

8.3. Conclusion

To summarize the observations, the local memory pattern has clear benefits in utilization of the GPU on larger array sizes. The available bandwidth & memory utilization is higher when compared to the global pattern. For smaller array sizes, there is not discernible speedup, as shown in Figure 18.

Thus, the compute size should be taking into account when deciding which pattern to use. Further experimentation with improved kernels (as presented in exercise set) is warranted to further analyze local memory behavior.

9. Appendix

9.1. Microbenchmark

9.1.1. Desktop

NVIDIA GeForce RTX 3070 (platform NVIDIA CUDA)

Expert level: Expert (3)

microbenchmarks measuring computational performance				
measure all	Perf (GOps)	I (c)	L (c)	r (warps)
SP	6370	0.42	2.77	128
SP/2 indep	7460	0.358	1.33	64
SP/4 indep	8190	0.326	0.803	160
SP/8 indep	4400	0.607	1.06	24
MADD	13600	0.197	1.16	128
MADD/2 indep	15900	0.168	0.683	64
MADD/4 indep	17400	0.153	0.517	160
INT	4460	0.599	2.33	128
INT/2 indep	4880	0.547	1.33	64
INT/4 indep	5140	0.519	1.33	64
INT/8 indep	1340	1.99	2.59	8
IMADD	8830	0.303	1.26	128
IMADD/2 indep	9680	0.276	0.674	64
IMADD/4 indep	10300	0.259	0.466	64
DIV	3050	0.876	3.36	64
DIV/2 indep	3280	0.814	2.51	96
DIV/4 indep	3350	0.798	1.47	48
DIV/8 indep	3330	0.813	1.09	12
DP	37.8	70.6	79.4	3
DP/2 indep	38	70.5	78.1	2
DP/4 indep	38.3	69.8	71.5	2
DP/8 indep	...	...	...	...
SF	686	3.89	10.7	32
SF/2 indep	1400	1.9	2.97	24
SF/4 indep	2770	0.964	1.39	16
SFSOFT	214	12.6	44.4	24
SFSOFT/2 indep	256	10.5	38.8	160
SFSOFT/4 indep	261	10.3	44.4	24
ATAN	...	...	...	...
CMP	2530	1.06	5.61	64
CMP/2 indep	2630	1.01	1.83	32
MOV	...	...	...	...
KERNEL	1510	1.77	1.77	-
GLOBALID	185	14.5	510	320
LOCALID	204	13.1	435	352

Figure 19: Desktop - Microbenchmark Results 1

NVIDIA GeForce RTX 3070 (platform NVIDIA CUDA)

Expert level: Expert (3)

CMP/2 indep	2630	1.01	1.83	32	
MOV	...	...	...	...	
KERNEL	1510	1.77	1.77	-	
GLOBALID	185	14.5	510	320	
LOCALID	204	13.1	435	352	
microbenchmarks measuring memory performance					
measure all	BW (GBs)	I (c)	L (c)	r (warps)	size (KB)
Global/Float/MainMemory	204	-	-	-	-
Global/Float/CacheLevel1	1010	-	-	-	32
Global/Float/CacheLevel2	...	...	...	...	...
Global/Char/MainMemory					
Global/Char/CacheLevel1					
Global/Char/CacheLevel2					
Global/Float2/MainMemory					
Global/Float2/CacheLevel1					
Global/Float2/CacheLevel2					
Local/Float/MainMemory	...	...	...	...	...
Local/Float/CacheLevel1	3160	-	-	-	-
Local/Char/MainMemory					
Local/Char/CacheLevel1					
Local/Float2/MainMemory					
Local/Float2/CacheLevel1					
Private/Float/MainMemory	...	...	...	...	...
Private/Float/CacheLevel1	2080	5.22	20.3	-	-
Private/Char/MainMemory					
Private/Char/CacheLevel1					
Private/Float2/MainMemory					
Private/Float2/CacheLevel1					
Constant/Float/MainMemory	170	-	-	64	-
Constant/Float/CacheLevel1	178	60.1	183	-	-
Constant/Char/MainMemory					
Constant/Char/CacheLevel1					
Constant/Float2/MainMemory					
Constant/Float2/CacheLevel1					

microbenchmarks measuring specific features	
measure all	Estimated parameter
WARPSIZE	Size of a hardware thread (warp) = - work items
ROOFLINE_MODEL	Ridge point of Computational Intensity = 1024 ops/byte
MATRIX_MULTIPLICATION	Maximal computational performance = 0.01 GOps
VECTOR_OPERATIONS	Maximal memory bandwidth = 440 GBs
Memory access	...

Figure 20: Desktop - Microbenchmark Results 2

9.2. Specifications

9.2.1. Desktop

Part	Value
CPU	Ryzen 9 5950X
GPU	RTX 3070
OpenCL	300 (Set in code)
Driver Version	591.59
RAM	64GB (3200 Mhz)
OS	Windows 11 - Version 10.0.22631 Build 22631

Table 1: Desktop Specifications

```
→ .\deviceQuery.exe
Platform 0
    Name:       NVIDIA CUDA
    Vendor:     NVIDIA Corporation
    Profile:    FULL_PROFILE
    Version:    OpenCL 3.0 CUDA 13.2.73

    Device: 0
        Name                        : NVIDIA GeForce RTX 3070
        Device Type                 : CL_DEVICE_TYPE_GPU
        Device Type                 : NVIDIA Corporation
        OpenCL C Version            : OpenCL C 1.2
        #Compute Units (cores)      : 46
        Native vector width         : 1
        2D Image limits             : 32768x32768
        3D Image limits             : 16384x16384x32768
        Global memory size [MB]     : 8191
        Global memory cache size [KB] : 1288
        Local memory size [KB]      : 48
        Maximum buffer size [MB]    : 2047
        Maximum workgroup size      : 1024
        Timer resolution            : 1000
        Clock frequency  [MHz]      : 1815
        Cache Type                  : CL_READ_WRITE_CACHE
        Compute capability          : 8.6
        Nvidia architecture         : Ampere
        Registers per SM (32-bit)   : 65536
        Warp size                   : 32
        Integrated Memory           : No
```

Figure 21: Desktop RTX3070 Device Query

9.3. Charts

9.3.1. Part 1

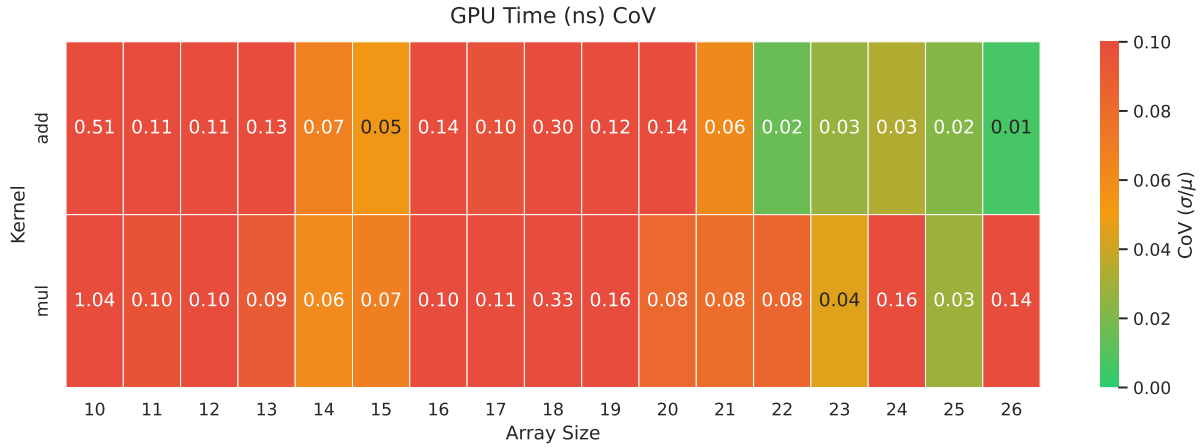

Figure 22: Desktop - Part 1 - GPU Time CoV

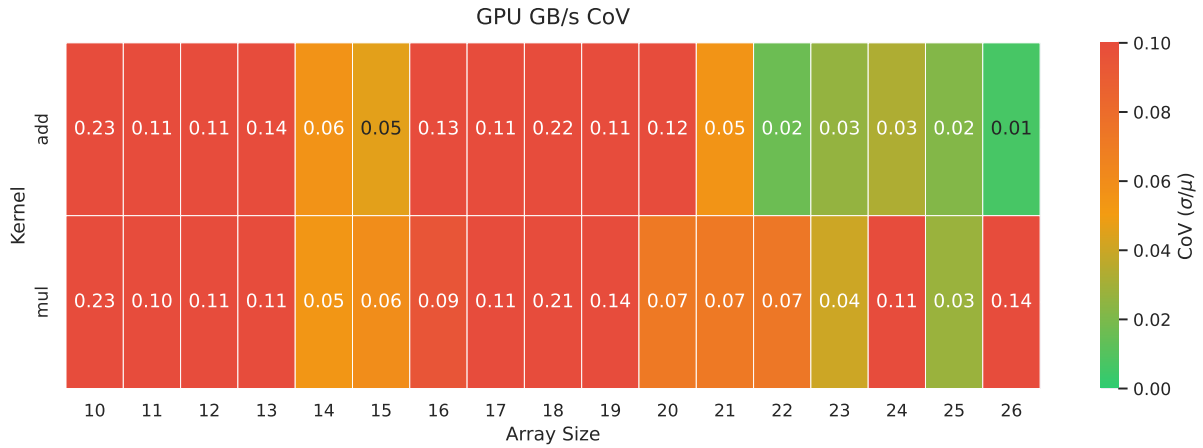

Figure 23: Desktop - Part 1 - Memory Bandwidth CoV

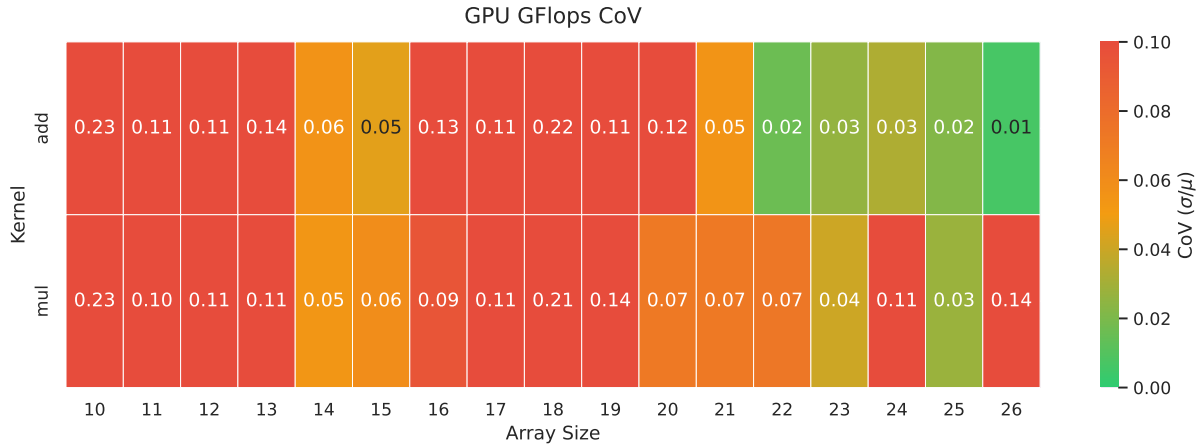

Figure 24: Desktop - Part 1 - Compute Throughput CoV

9.3.2. Part 2

Figure 25: Desktop - Part 2 - GPU Time CoV

9.3.3. Part 3

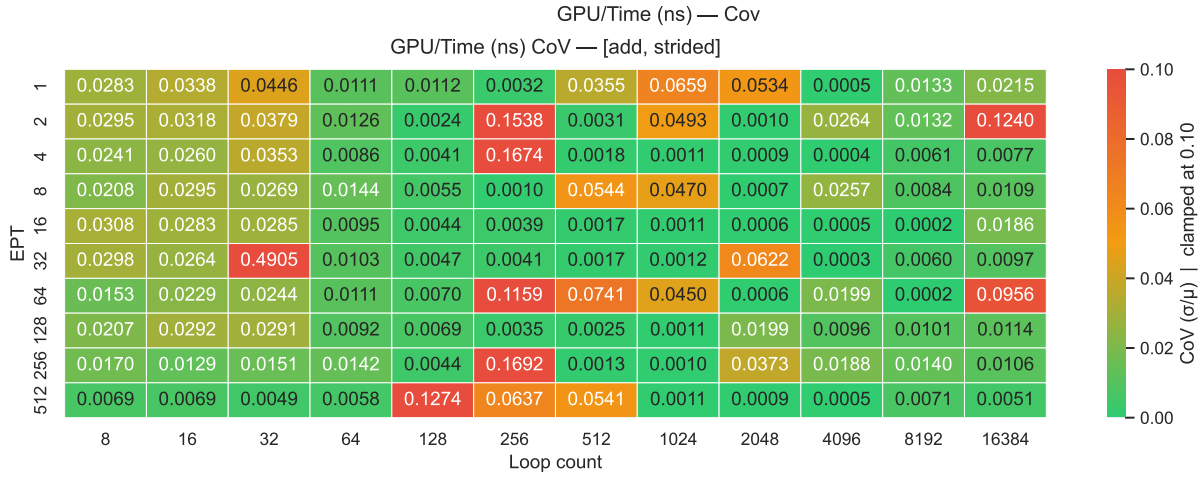

Figure 26: Desktop - Part 3 - GPU Time CoV

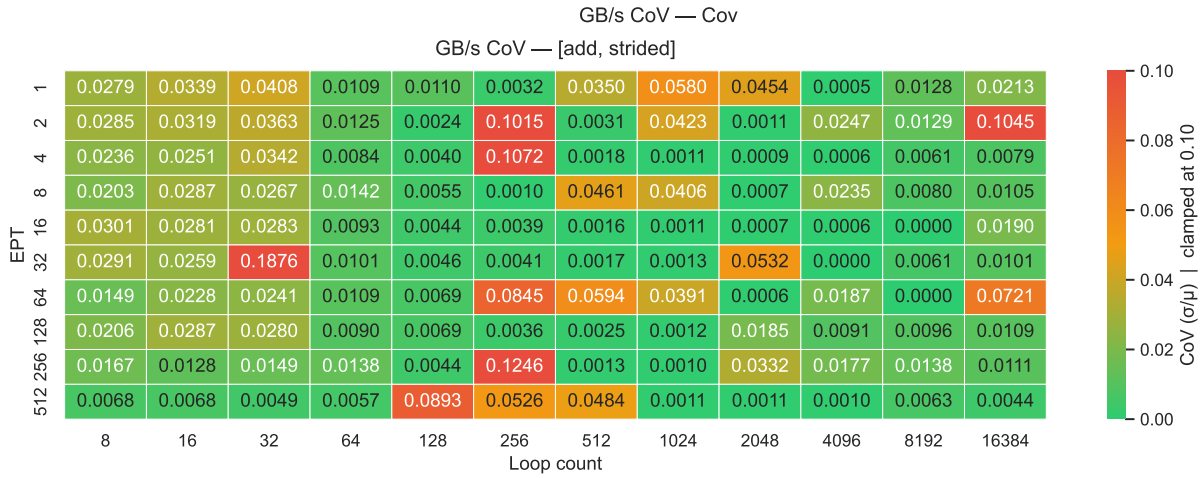

Figure 27: Desktop - Part “” - Memory Bandwidth CoV

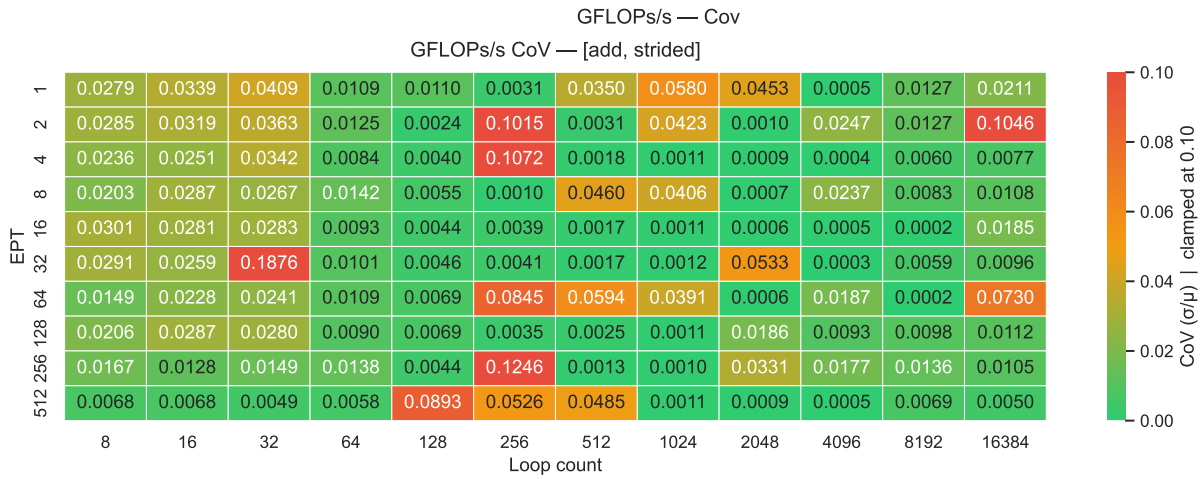

Figure 28: Desktop - Part 3 - Compute Throughput CoV

9.3.4. Part 3-2

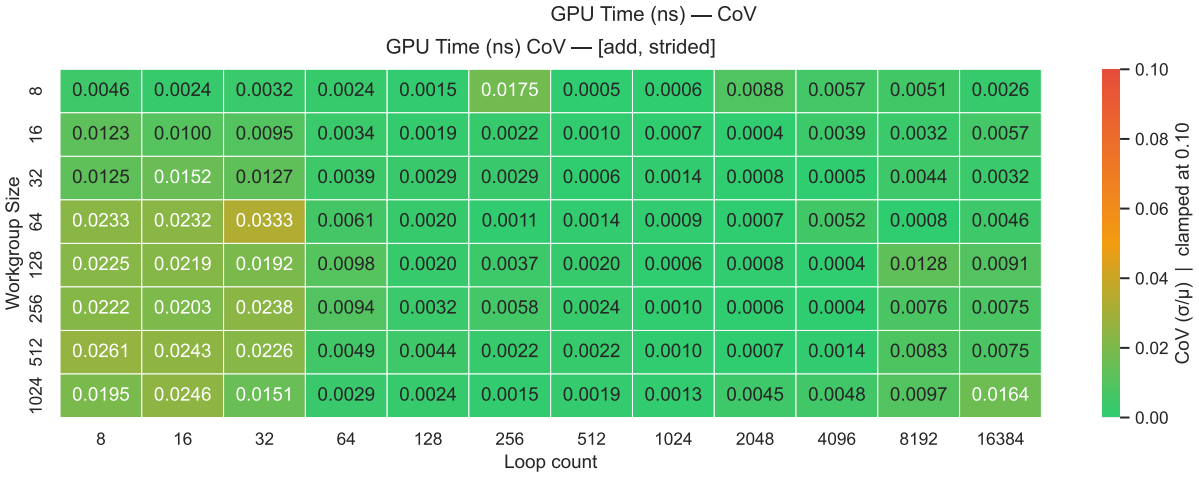

Figure 29: Desktop - Part 3.2 - GPU Time CoV

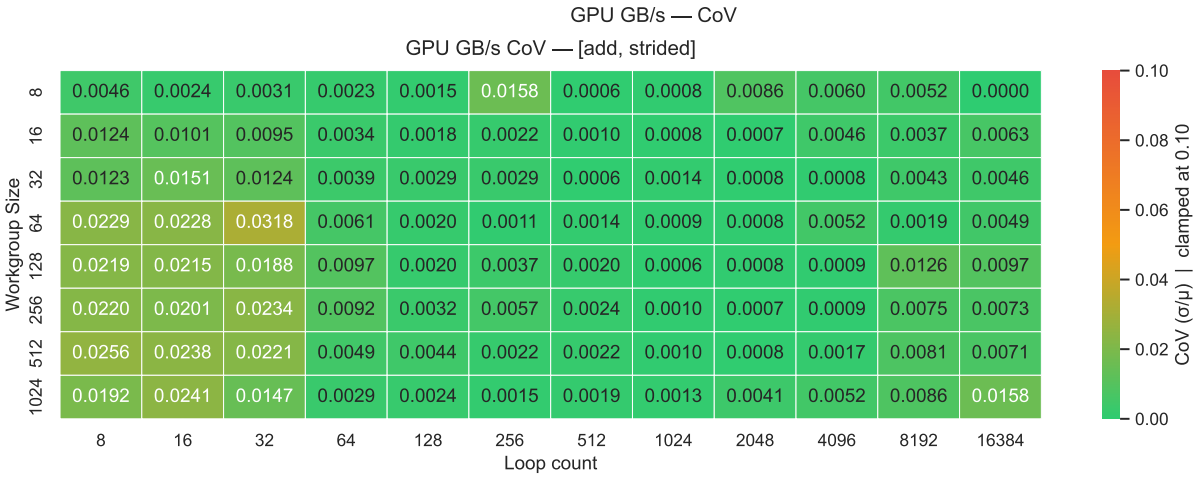

Figure 30: Desktop - Part 3.2 - Memory Bandwidth CoV

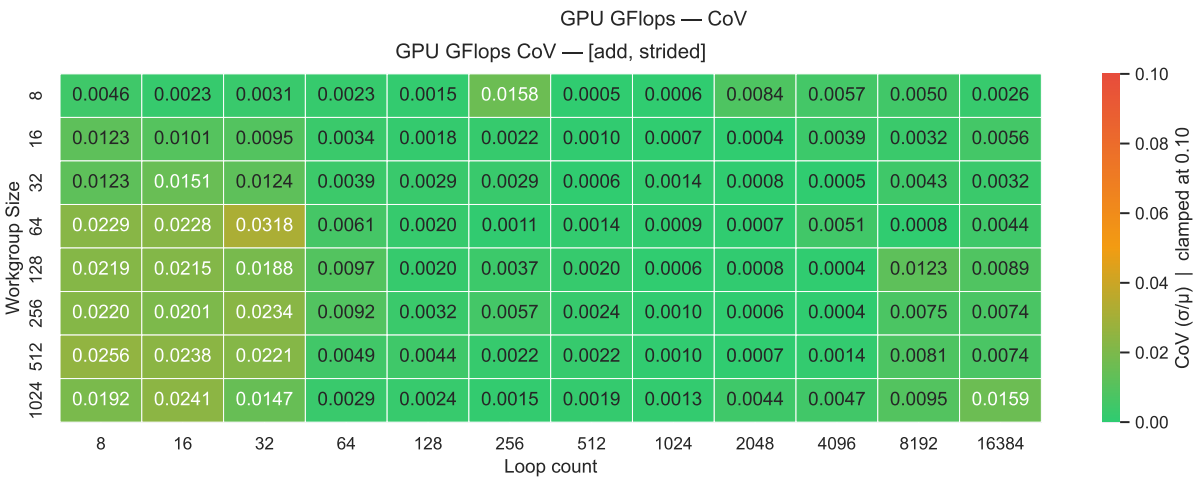

Figure 31: Desktop - Part 3.2 - Compute Throughput CoV

9.3.5. Part 4

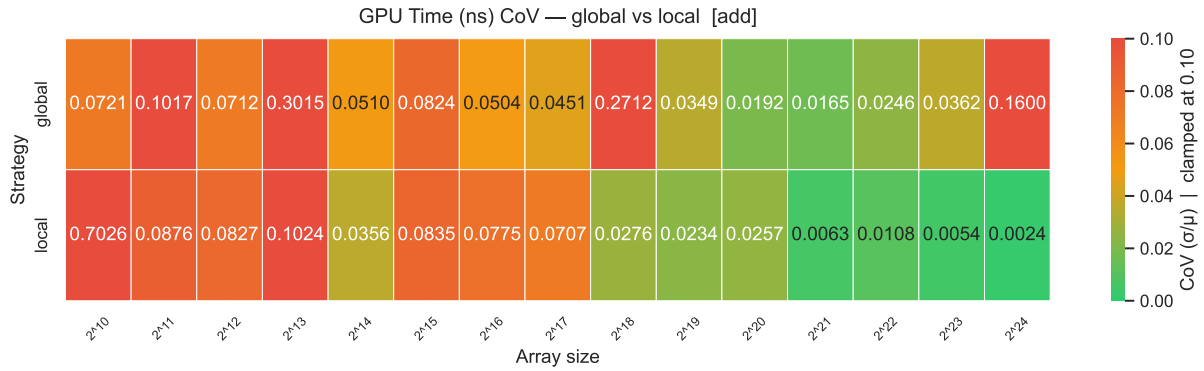

Figure 32: Desktop - Part 4 - GPU Time CoV

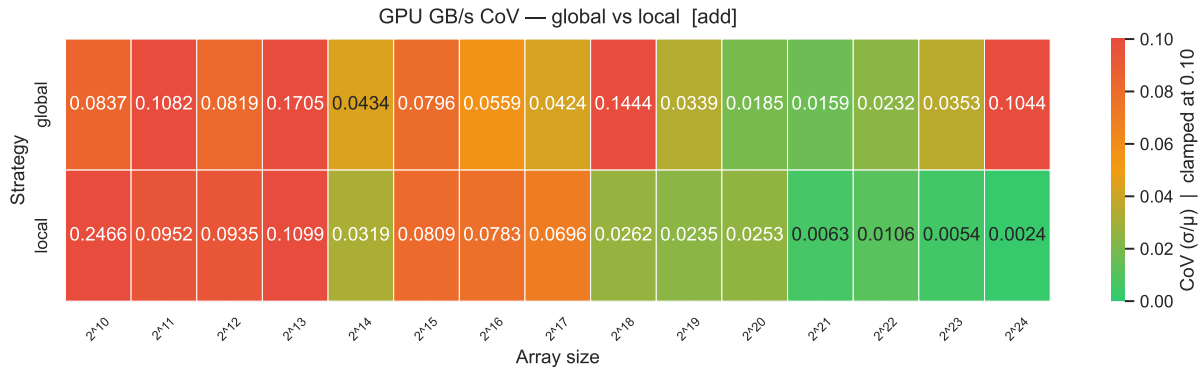

Figure 33: Desktop - Part 4 - Memory Bandwidth CoV

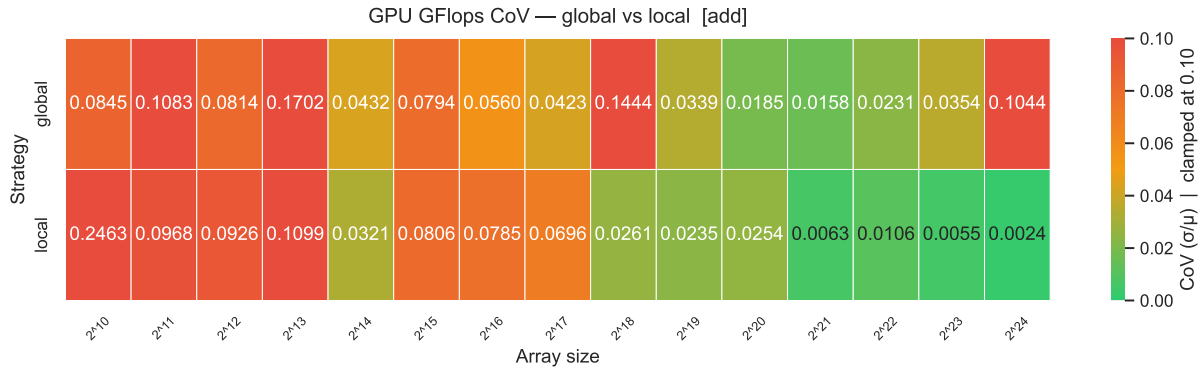

Figure 34: Desktop - Part 4 - Compute Throughput CoV

Bibliography

[1] J. Lemeire, “Lesson 2: Programming GPUs.” [Online]. Available: http://parallel.vub.ac.be/education/gpu/theory/GPU%20Computing%20-%20Lesson%202%20doc%20-%20Programming%20GPUs%20-%20levels%200%201%20and%202.pdf°

[2] J. Lemeire, “Mini Project - Part 4 - Schema.” [Online]. Available: http://parallel.vub.ac.be/education/gpu/labs/miniproject_step4_with_local%20memory.jpg°

[3] P. A. Paolillo, “Lecture 2 - Metrics, Distributions & Experimental Rigor - How Many runs do I need?.” p. 38, 2025.

[4] P. A. Paolillo, “Lecture 2 - Metrics, Distributions & Experimental Rigor - Coefficient of Variation (CoV).” p. 25, 2025.